AI Academy, National AI Center

MLE-AI-201 – Machine Learning – Fall 2026

# Pricing Baku Homes from Scratch

## A NumPy Decision Tree and Pegasos SVM on bina.az Listings

Farhad Budagov • Murad Atlukhanov • Panah Guluyev • Mikayil Hasanov

*Final Project Defense – 18 October 2026*

`github.com/pnah6699-maker/bina-az`

# The problem in one sentence

## What we built

Given a bina.az sale listing (size, rooms, floor, location, condition), predict its **price** and whether it is a **premium** or **standard** home — with a decision tree and an SVM that we wrote *from scratch in NumPy*, benchmarked honestly against scikit-learn.

**Task A — Regression**

- Target: $\log(\text{price})$ (price skew 5.5 $\rightarrow$ 0.29)
- Model: our decision tree (MSE)
- Metrics: RMSE, MAE, $R^2$, MAPE

**Task B — Classification**

- Premium = price $>$ **train** median (209,000 AZN)
- Models: our tree (Gini/entropy), Pegasos SVM, RFF-SVM
- Metrics: precision, recall, F1, ROC-AUC

**One command reproduces everything:** `python -m src.run_all`

# Getting the data from Kaggle

**Source**

- Kaggle: `sehriyarmemmedli/binaaz-sale-project` (version 1)
- `house_sale.csv`: **100,775 rows** × **51 columns**, 142 MB
- Scraped from bina.az, 5–16 October 2024, all prices in AZN
- Azerbaijani column names: *Sahə*, *Otaq sayı*, *Mərtəbə*, *Təmir* ...

**Not committed to Git** (`data/` is in `.gitignore`); every member downloads it locally.

**How we download it** (no API key needed)

```
# scripts/download_data.py
import kagglehub, shutil
path = kagglehub.dataset_download(
  "sehriyarmemmedli/binaaz-sale-project")
shutil.copyfile(path + "/house_sale.csv",
                "data/house_sale.csv")
```

```
$ python scripts/download_data.py
```

# Cleaning a messy scrape — and removing leakage

## Cleaning funnel

| Step | Rows |
|---|---|
| Raw dump | 100,775 |
| Keep latest scrape per listing URL | 64,454 |
| Residential only (new / old / house) | 58,452 |
| Drop exact re-posts | 57,699 |
| Price 10k–5M AZN, area 15–1000 $m^2$ | 57,594 |
| Price/$m^2$ sanity, coords in AZ | **57,473** |

Parsed `"145 m²"`, `"4.5 sot"`, `"7 / 9"`; median imputation + missing flags → **92 features**.

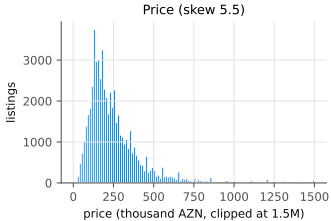
### Leakage removed

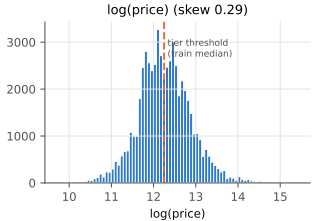
- `unit_price` = price / area
- `total_price` = price
- IDs, owner names, address, `views`

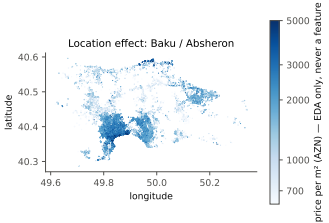
### No test-set peeking

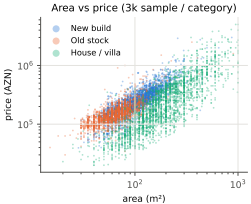
Featurizer, tier threshold and scaler are fit on **train only** (and re-fit inside every CV fold). Split 70/15/15, stratified.

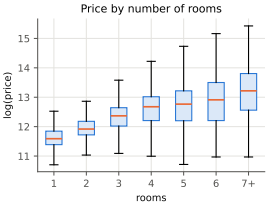
# What the data looks like

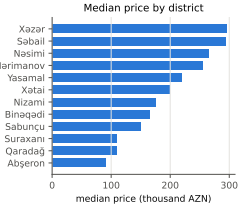


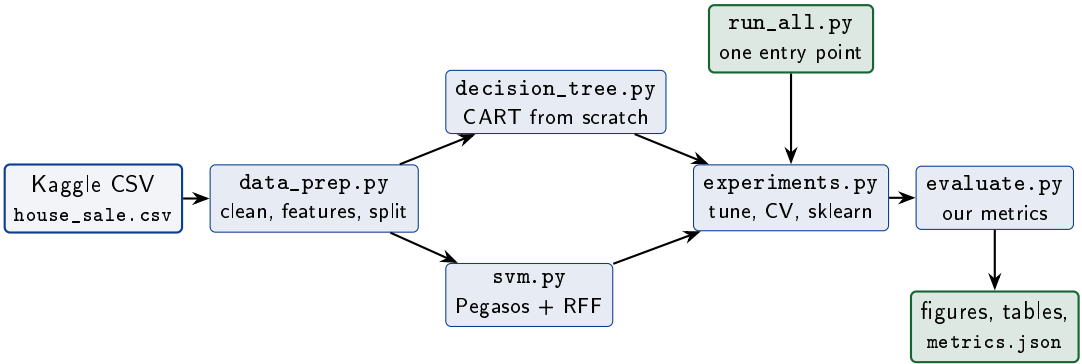



# Pipeline: one command, every number




Blue boxes = our code. scikit-learn appears **only** as a baseline inside `experiments.py` and in the tests.

Full run: 11 minutes on a laptop CPU, seed 42.

# Our decision tree (NumPy only)

## How it grows

- Impurity: Gini $1 - \sum p_k^2$, entropy $-\sum p_k \log_2 p_k$, MSE
- Split = feature/threshold with the largest gain $IG = I(\text{parent}) - \sum_c \frac{N_c}{N} I(c)$
- Sort each feature **once**; score all cut points with cumulative sums
- Stops: `max_depth`, `min_samples_split`, `min_samples_leaf`, `min_impurity_decrease`
- Leaf = mean or class frequencies (`predict_proba`)

## Does it match scikit-learn?

- Toy 8-row example: **identical** splits & thresholds
- Real data: same depth, leaves $\pm 4$, **94.5% / 98.9%** identical predictions
- Differences = **ties**: ours `area`$\leq$100.1, sklearn `log_area`$\leq$4.606 — same split, since $e^{4.606} = 100.1$

# Our SVM: Pegasos sub-gradient descent

$$J(\mathbf{w}) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\textstyle\sum_i \max(0, 1 - y_i\,\mathbf{w}^\top \mathbf{x}_i)$$

```
eta = 1.0 / (lam * t)           # step size
active = yb * (Xb @ wb) < 1.0   # wrong / in margin
grad = (yb[active, None] * Xb[active]).sum(0)/k
wb *= (1.0 - eta * lam)         # shrink: wider margin
wb += eta * grad                # fix the mistakes
```

- Mini-batches of 32, 30 epochs, bias folded into $\mathbf{w}$
- $J$: 1.00 $\rightarrow$ 0.270 (1 epoch) $\rightarrow$ 0.256

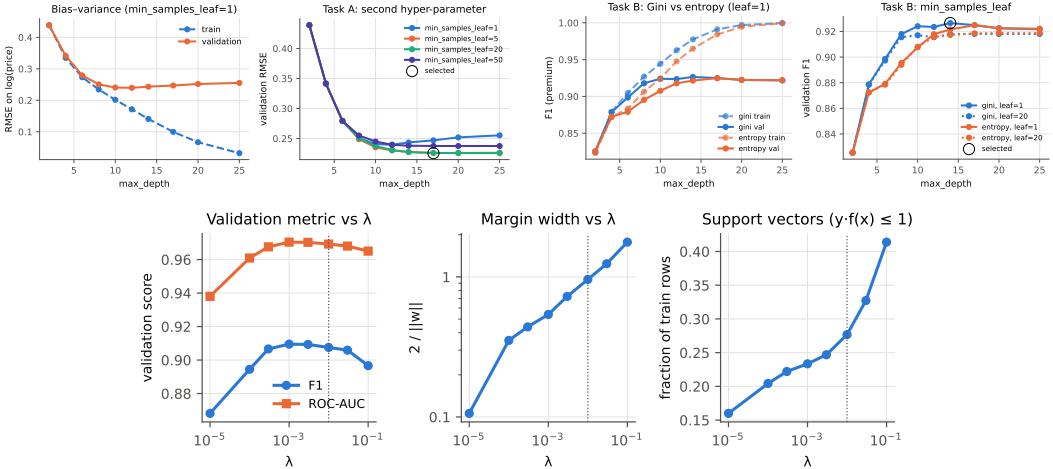
## A margin you can see ($\lambda = 0.01$)

$\|\mathbf{w}\| = 2.09$, margin $2/\|\mathbf{w}\| = 0.96$
Support vectors: **11,135** (27.7%)

## Non-linear: random Fourier features

$z(\mathbf{x}) = \sqrt{2/D}\cos(W\mathbf{x} + c)$,
$D = 1000$, then the same linear solver.
Chosen over kernel Pegasos: $O(D)$ vs. $O(n)$ per step on 40k rows.

# Tuning on validation only: bias vs. variance



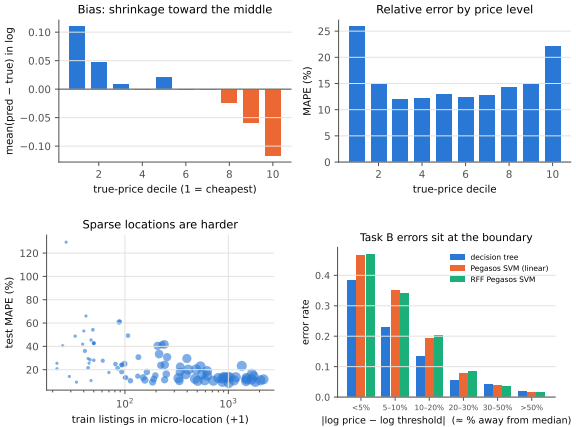

Selected: tree A depth 17 / leaf 5; tree B Gini, depth 14; SVM $\lambda = 0.01$ (larger $\lambda \Rightarrow$ wider margin, more support vectors).

# Results: test set (5-fold CV mean ± std)

| Model | Task A RMSE (log) | Task B F1 | ROC-AUC |
|---|---|---|---|
| **Ours: decision tree** | **0.232** (0.232±0.004) | **0.921** (0.926±0.004) | 0.917 |
| sklearn DecisionTree | 0.232 (0.233±0.004) | 0.921 (0.926±0.004) | 0.916 |
| **Ours: Pegasos SVM** | — | **0.900** (0.907±0.003) | 0.965 |
| sklearn SGD hinge | — | 0.900 (0.908±0.003) | 0.965 |
| **Ours: RFF SVM** | — | 0.900 (0.906±0.003) | 0.967 |
| Mean / majority baseline | 0.629 | 0.000 | 0.500 |

- Task A: $R^2 = 0.864$, MAPE 15.4%, MAE $\approx$ 44.6k AZN
- Ours = sklearn within **0.002** on every metric; ours is $\sim$10$\times$ slower (6.6 s vs 0.5 s)
- Tree wins F1, SVM wins AUC: leaf frequencies are coarse scores, the SVM ranks smoothly

# Where the models fail




- **Luxury & bargain outliers:** cheapest decile over-predicted, most expensive under-predicted (MAPE 26% / 22% vs. 12% mid)
- **Houses/villas:** 32% MAPE vs. 12% for flats
- **Sparse locations** (<150 train rows): 29.9% vs. 14.3% MAPE
- **Tier errors at the boundary:** 38–46% wrong within 5% of the median, <2% beyond 50%
- Worst cases: instalments listed as the price

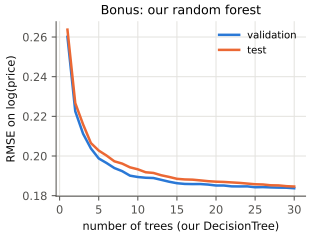
# Bonus work

**P8 — Ridge (normal equations)**

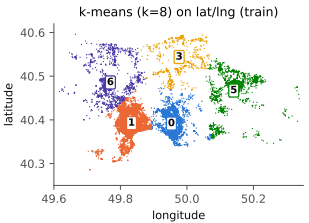
- $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$
- RMSE 0.242; coefficients $=$ sklearn to $10^{-10}$

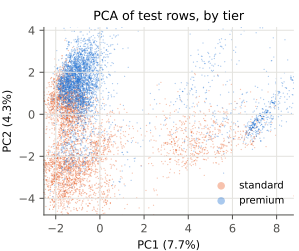
**P9 — Random forest of our trees**

- 30 trees, bootstrap + 33% features
- RMSE **0.264** $\rightarrow$ **0.185**; sklearn: 0.185




**P10 — k-means & PCA**



# Testing & reproducibility

**40 unit tests, all passing** (`pytest`)

- Tree splits = sklearn on a fully determined toy example
- Every stopping rule changes the tree
- SVM objective decreases; larger $\lambda \Rightarrow$ wider margin
- RFF approximates the RBF kernel and solves XOR
- Our metrics = `sklearn.metrics`; leakage columns dropped

**From a clean checkout**

```
$ pip install -r requirements.txt
$ python scripts/download_data.py
$ python -m src.run_all
$ python -m pytest -q
```

- Pinned versions, seed 42 everywhere
- Tables in the report are generated by code

# Limitations & what we would improve

**Where the project is weak today**

- *Asking* prices from an 11-day snapshot — not real transaction prices
- Fixed outlier rules miss some instalment prices and drop villas above 5M AZN
- Free-text descriptions (renovation, documents, view) are unused
- Our NumPy tree is $\sim 10\times$ slower than compiled code $\Rightarrow$ less forest tuning

**Given another week we would**

1. Add text features from the listing descriptions
2. Use a time-based split, closer to real deployment
3. Target-encode sparse micro-locations (biggest error source)
4. Build gradient boosting from our own trees

# Team contributions

| Member | Primary ownership | Code share |
|---|---|---|
| Farhad Budagov | Decision tree + metrics (`decision_tree.py`, `evaluate.py`, tests) | 26% |
| Murad Atlukhanov | Data cleaning, features, EDA (`data_prep.py`, `eda.py`, `plots.py`, tests) | 24% |
| Panah Guluyev | Pegasos SVM + RFF, bonus models, pipeline (`svm.py`, `ensemble.py`, `linear.py`, `run_all.py`) | 26% |
| Mikayil Hasanov | Experiments: tuning, sklearn benchmark, CV, error analysis (`experiments.py`) | 24% |

**We can all answer:**

Any of us can walk through any module on demand. Pick a file — we'll defend it.

**AI tool use:** Claude Code (Anthropic) drafted code, tests and report text; every member reviewed, ran and can explain their part (disclosed in the report).

# Questions?

---

Farhad Budagov • Murad Atlukhanov • Panah Guluyev • Mikayil Hasanov

`github.com/pnah6699-maker/bina-az`

*"We can walk through any file in the repo."*